1.Software Architecture

The overall project includes 10 java files, of which CentralArea Menu NoFlyZones Order Restaurant is the entity class corresponding to the data provided by Rest
HttoClient is a network tool class, initialized by jackson to achieve the server to get data and transformed into JAVA entity classes
Main class provides the entrance to the program and calls HttoClient to initialize the data and achieve the function of invalid order filtering.
LngLat is the latitude and longitude object of a point on the map, and is also the object corresponding to a single node in A star.
The AStar class is the core implementation of the path planning algorithm, which encapsulates the specific implementation details of the A Star algorithm for Main to use.
For OrderOutcome order results and MoveDirection flight angle use an enumeration class to store the relevant constants.
The overall program uses the A* algorithm as the implementation of the path planning algorithm, which has more efficient features than other path planning algorithms. The key to calculate the squares that make up the path is the following equation (heuristic function) :

$$F=G+H$$

The algorithm will give priority to traverse the node that tends to the end point, with good efficiency, tested single path planning from Appleton Tower to restaurant computing time is about 500ms
At the same time, the path from the restaurant to the starting point and the number of drone movements required for a single trip are cached in the Restaurant. This is used for direct recall of the next orders. Although this adds some computation in the case of very small dispatch orders, it greatly reduces the number of path planning sessions in common situations.

2.Classes

The following section contains a brief description of all the classes used in practice. Getter" and "setter" are omitted from the documentation because their functions are trivial and their inclusion in this document only makes them more confusing.
They are not important and their inclusion in this document would only make them more confusing.

2.1 AStar

A*Pathfinding algorithm

2.1.1 attribute

private static PriorityQueue open
Minimum heap
private LngLat last
Record the last node

private LngLat endLngLat
Record the last node
Map<String,LngLat> historyMap
History node list is close list
private List<List> noFlyList
No Fly Zone node list

2.1.2Method

public LngLat start(LngLat startLngLat, LngLat endLngLat, List<List> noFlyList){
Start A* search, the specific implementation is
initialize the two Lists open_set and close_set.
add the starting point to open_set.
Private void search()
Start the search
If open_set is not empty, select the node n with the highest priority from open_set.
If node n is the end point, then:
-tracks the parent node step by step from the end point until it reaches the start point.
-return the found result path and the algorithm ends.
If node n is not the end point, then.
-removes node n from the open_set and adds it to the close_set.
-Iterate through all neighboring nodes of node n according to the direction enumeration class.
-If neighboring node m is in close_set, then:
-skip and iterate through the next neighboring node
-If neighboring node m is also not in open_set, then:
Set the parent of node m to node n
Calculate the priority of node m
Add node m to the open_set
private LngLat printPath()
Get the path (final node)

2.2CentralArea

2.2.1 attribute

private String name
Location name
private Double longitude
Latitude
private Double latitude
Longitude

2.3 HttpClient

Network acquisition tool class

2.3.1 attribute

private static volatile HttpClient instance;
ensure that instance is synchronized in all threads
private static String baseUrl = "https://ilp-rest.azurewebsites.net/";
initialize url
private static ObjectMapper objectMapper
jackson ObjectMapper object

2.3.2 method

public static synchronized HttpClient getInstance()
return HttpClient object, used for subsequent network operations
public static List getJsonToList(String echoBasis, Class beanType)
According to the path to get Rest server Json data to List
public static List getJsonToList(URL url, Class beanType)
Directly according to the url to get Rest server Json data to List
public static String getJson(String echoBasis)
According to the path to get Rest server Json data
public static String getJsonByMap(Object o)
map to json

2.4 LngLat

Coordinate node

2.4.1 attribute

public static final Double MINIMUMDISTANCE
Less than the distance is considered to be in the vicinity
public static final Double MOVEUNITLENGTH
Minimum moving distance
private double F
F = G + H minimum heap or priority queue of sorted points, the smaller the value the more priority search
private double H
indicates the cost of moving from the starting point A to the specified square on the grid (can be moved in the diagonal direction)
private double G;
G represents the cost of moving from starting point A to the specified square on the grid (can be moved in diagonal direction)
private LngLat parent;
The parent of the node
private MoveDirection parentMoveDirection
the flight angle from the parent node to the node
private Double lat;
Latitude
private Double lng
Longitude

2.4.2 method

public LngLat(Double lng, Double lat)
Node coordinates object initialization function
public LngLat(double F,double G,double H,Double lng,Double lat,LngLat parent){
Node coordinate object initialization function
public String getCoordinate()
Use the coordinates into String objects, used to de-duplicate, to prevent a single node is added multiple times open
public Boolean inCentralArea()
Whether in the central area
public Double distanceTo(LngLat lngLat)
Calculate the distance of this coordinate to the target node
public Boolean closeTo(LngLat lngLat)
determine whether the target coordinates have reached (according to the document when the helicopter distance from the target is less than 0.00015 can be considered to have reached the target near)
public LngLat nextPosition(MoveDirection direction, LngLat g,LngLat endLngLat)
According to the direction of movement (16 directions) to obtain the coordinates of the next node after the movement (latitude and longitude)
public boolean isPolygonContainsPoint(List lngLats)
Returns whether the point is within the polygon area (including polygon boundaries).
Single-sided ray method is used to make horizontal rays to the right, if the intersection is an odd number of points, it is located in the polygon.
Not only can determine the center of the image rectangle, but also can determine the irregular polygons similar to the no-fly zone
public int compareTo(Object o)
Comparison method called by the minimum heap Compare the value of F

2.5 Main

2.5.1 attribute

private static final String AppletonTowerName
Appleton Tower Name
private static List restaurants
List restaurants
private static String baseUrl
Rest server address
private static String orderDate
orderDate
private static int flyCount
Number of drones flying in a day
private static Set passRestaurant
The restaurants that the drone flies past

private static FeatureCollection featureCollectionFromJson
geojson object

2.5.2 method

public static void main(String[] args)
Control class and function entry
private static void saveDron()
Save drone flight geojson records
private static void initData()
Initialize data
private static void outputToFile(String json, String path)
Output a string to a file
private static OrderOutcome checkOrder(Order order)
Check order
private static OrderOutcome tryDeliveredOrder(Order order, Restaurant restaurant)
tryDeliveredOrder
private static void checkParms(String[] args)
Check the program entry parameters
public static Boolean checkCardNumber(String s)
Directly verify whether the pure number

2.6 Menu

Menu corresponding entity class

2.6.1 attribute

private String name
Name
private String priceInPence
price

2.7 MoveDirection

Drone flight direction enumeration class

2.8 NoFlyZones

No Fly Zone entity class

2.8.1 attribute

private String name
Name of the zone
private List<List> coordinates
Area boundary points

2.9 Order

Order entity class

2.9.1 attribute

private String orderNo;
orderNo
private Date orderDate
orderDate
private String customer
customer
private String creditCardNumber
credit card number
private String creditCardExpiry
credit card expiry
private String cvv
private Integer priceTotalInPence
price Total InPence
private List orderItems;
Order list

2.10 OrderOutcome

Order type enumeration class

2.11 Restaurant

Restaurant entity class

2.11.1 attribute

private String name;
private Double longitude ;
private Double latitude ;
public List menus;
Menu list
private List routes;
Flight paths
private List linepaths = new ArrayList<>();
Flight paths for use by geojson
private List linepathMaps = new ArrayList<>();
Flightpath flightpath use
public int flightsNumber;
The number of moves needed to fly to the restaurant

2.11.2 method

public static Restaurant[] getRestaurantsFromRestServer(URL serverBaseAddress)
Get the list of restaurants
public void setRoutes(LngLat routeEnd)
Parse the result path returned by A*

3. Algorithm

3. 1 Basic explanation

The drone is flying for path planning and needs to plan the shortest path while avoiding obstacles. The common path finding and traversal algorithms are breadth-first search and depth-first search, but in this assignment I am using the A algorithm for path planning, because in the case of more traversal directions, both depth-first search and breadth-first search require too many nodes to be traversed, while the A algorithm usually has better performance compared to the first two with the guidance of the heuristic function. The following is a simple comparison with A in order to use breadth-first search.

As its name suggests, breadth-first search searches with breadth as the priority.

Starting from the starting point, it first traverses the points around the starting point, then traverses the points adjacent to the points already traversed, and gradually spreads outward until the end point is found.

This kind of traversal is non-differentiated in all directions, and a large number of meaningless nodes are traversed before the target node is found. A*, on the other hand, has a good efficiency because the heuristic function gives priority to the nodes that tend to the end point.

Let us explain the heuristic function F=G+H

G is the cost of moving from the starting point A to the specified square on the grid (can be moved in the diagonal direction, the cost of the diagonal direction is the length of the diagonal)

H is the estimated cost of moving from the specified square to the end point B (there are many ways to calculate H, here we set it to move up, down, left and right only).

The implementation logic is as follows

initialize the open_set and close_set Lists.

add the starting point to open_set.

If open_set is not empty, select the node n with the highest priority from open_set.

If node n is the end point, then.

- Tracing the parent node step by step from the end point until it reaches the starting point.
- return the found result path and the algorithm ends.

If node n is not the end point.

- remove node n from the open_set and add it to the close_set.
- Iterate through all neighboring nodes of node n according to the direction enumeration class.
- If neighboring node m is in close_set.
- skip and iterate through the next neighboring node
- If neighboring node m is also not in open_set, then.
- Set the parent of node m to node n
- Calculate the priority of node m
- Add node m to open_set

3.2 Bug fixed

Since multiple path planning share the same Astar object (encapsulating the concrete implementation of A* algorithm) instance, the path cache in it leads to coupling between multiple path planning, as shown in figure 1.

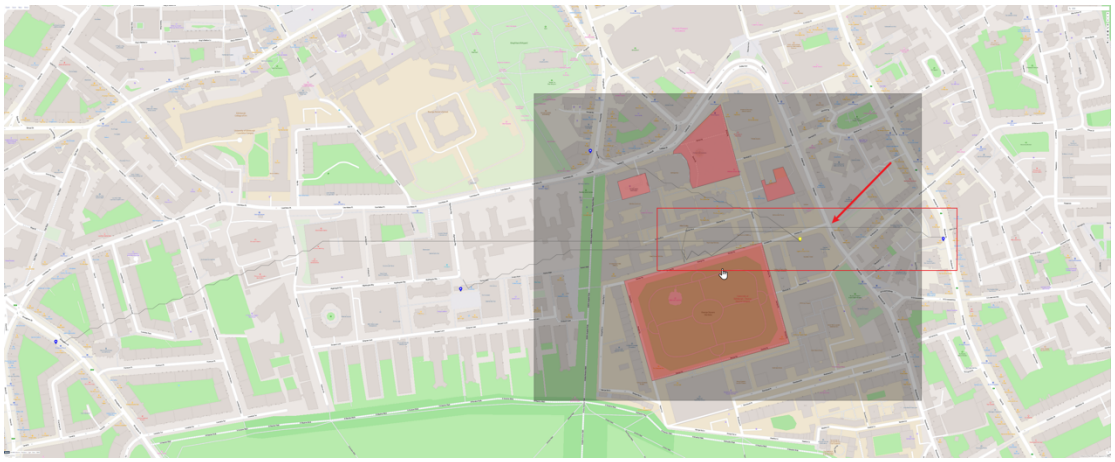

Figure 1

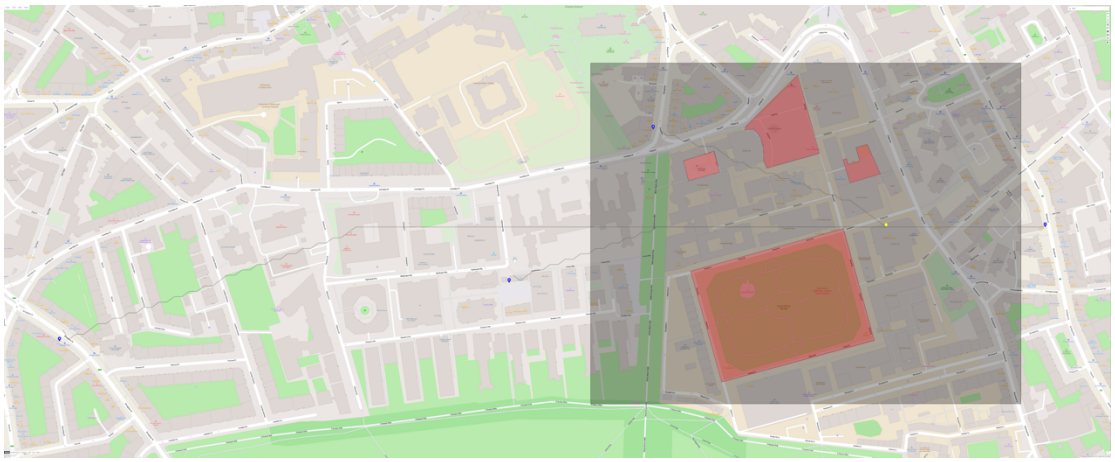

Figure 2

Figure 2 shows the result of planning after each planning with a separate Astar object.

3.3 Optimization points

In the third step, the node n with the highest priority is selected from open_set: if we use list implementation, we need to do an O(n) complexity traversal.

So here I use the PriorityQueue (the underlying logic is the minimum heap) for implementation, which greatly optimizes the time consuming path planning. As the planning length becomes longer, the optimization effect will be more obvious.

In the current test example, the node direct path planning time is generally controlled within 500ms.